

Lab 8

PSTAT 115, Fall 2025

Simple linear regression model

The simple linear regression model is:

$$y_i = \beta x_i + \alpha + \epsilon_i \quad i = 1, \dots, n$$

$\epsilon_i \sim \text{normal}(0, \sigma^2)$ and ϵ_i 's are independent.

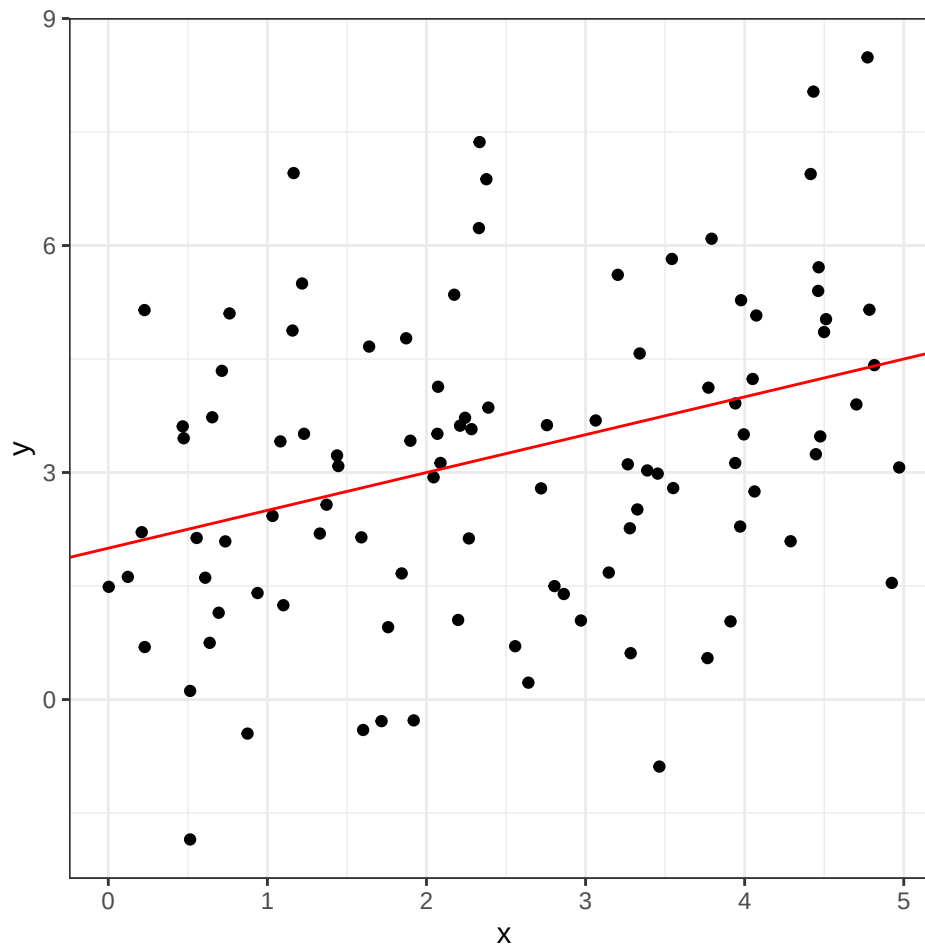
This model can be written as $y_i - \beta x_i - \alpha \sim \text{normal}(0, \sigma^2)$ which is equivalent to $y_i \sim \text{normal}(\beta x_i + \alpha, \sigma^2)$. We will use this last expression of the model to code simple linear regression with stan.

Note, our parameters are β, α and σ . In frequentist view, we find the likelihood and compute the MLE then mission's complete. But in Bayesian context, we can specify some priors, find the posterior distribution and estimate those parameters with posterior means/medians/modes etc. Of course, the posterior distribution might be complicated and we need to turn to MCMC for help. That's when we would like to use stan to tackle problems.

We code the model in a stan file as follows. Note: in stan, if you don't specify the prior, it will use the improper priors for the unbounded parameters.

```
data {  
  int<lower=0> N;  
  vector[N] x;  
  vector[N] y;  
}  
parameters {  
  real alpha;  
  real beta;  
  real<lower=0> sigma;  
}  
model {  
  y ~ normal(alpha + x*beta, sigma);  
}
```

```
set.seed(123)  
x <- runif(100, 0, 5)  
a <- 2  
b <- 0.5  
y <- b*x+a+rnorm(100, mean = 0, sd = 2)  
n <- length(y)  
ggplot(data = data.frame(x=x,y=y))+geom_point(aes(x=x,y=y))+geom_abline(slope = b, intercept = a, col =
```



```
stan_model2 = cmdstan_model("simple_linear_regression.stan")
stan_fit2 = stan_model2$sample(
  data = list(N = n, y = y, x=x),
  refresh = 0, show_messages=FALSE)
```

```
samples <- stan_fit2$draws(format = "df")
samples
```

```
## # A draws_df: 1000 iterations, 4 chains, and 4 variables
```

```
##   lp__ alpha beta sigma
```

```
## 1 -117  1.3 0.72  2.1
```

```
## 2 -118  3.0 0.21  1.9
```

```
## 3 -117  2.8 0.14  1.9
```

```
## 4 -117  2.6 0.18  2.0
```

```
## 5 -115  2.4 0.36  2.0
```

```
## 6 -116  2.4 0.42  2.0
```

```
## 7 -116  1.7 0.47  1.9
```

```
## 8 -115  1.9 0.47  1.9
```

```
## 9 -116  2.4 0.34  2.1
```

```
## 10 -116  1.5 0.66  1.9
```

```
## # ... with 3990 more draws
```

```
## # ... hidden reserved variables {'.chain', '.iteration', '.draw'}
```

```
alpha_samples <- samples$alpha
beta_samples <- samples$beta
sigma_samples <- samples$sigma
```

Now let's look at the estimates of posterior means.

```
mean(beta_samples)
```

```
## [1] 0.4632032
```

```
mean(alpha_samples)
```

```
## [1] 1.982226
```

```
mean(sigma_samples)
```

```
## [1] 1.960251
```

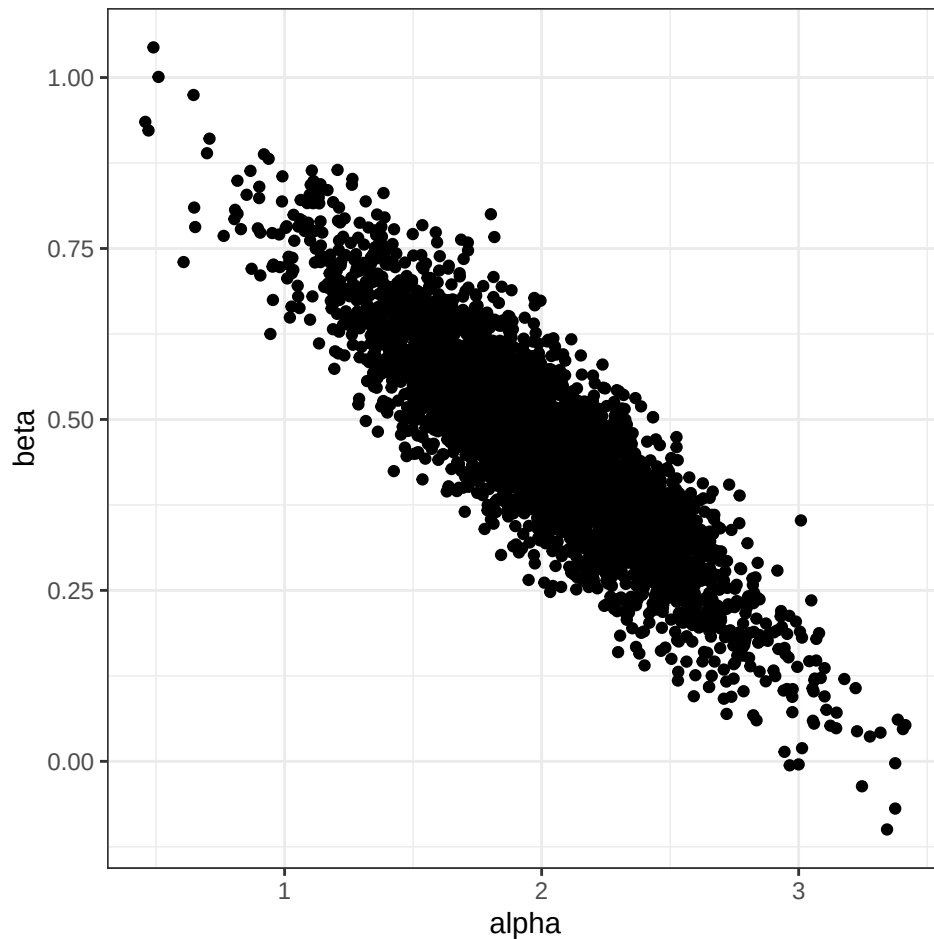
Let's say if we are interested in the predicted value when $x=5$, that is $\hat{y} = \alpha + 5\beta$. Then we can get samples of this predict value. The sample mean will give us a good estimate of the posterior mean of this predict value at $x = 5$.

```
y_hat <- alpha_samples + 5*beta_samples
mean(y_hat)
```

```
## [1] 4.298242
```

We can also explore the posterior relationship between α and β .

```
ggplot(tibble(alpha=alpha_samples, beta=beta_samples)) + geom_point(aes(x=alpha, y=beta)) + theme_bw()
```



Making 60% and 90% confidence bands plot for y

```
xgrid <- seq(0, 5, by=0.1)
```

```
xgrid
```

```
## [1] 0.0 0.1 0.2 0.3 0.4 0.5 0.6 0.7 0.8 0.9 1.0 1.1 1.2 1.3 1.4 1.5 1.6 1.7 1.8
## [20] 1.9 2.0 2.1 2.2 2.3 2.4 2.5 2.6 2.7 2.8 2.9 3.0 3.1 3.2 3.3 3.4 3.5 3.6 3.7
## [39] 3.8 3.9 4.0 4.1 4.2 4.3 4.4 4.5 4.6 4.7 4.8 4.9 5.0
```

```
compute_curve <- function(sample) {
  alpha <- sample[1]
  beta <- sample[2]
  y_values <- alpha + beta*xgrid
}
```

```
res <- apply(cbind(alpha_samples, beta_samples), 1, compute_curve)
```

```
#each col of res is for a set of alpha,beta values
```

```
#each row of res is different y values computed from different sets of alpha beta for a fixed x values.
```

```
quantiles <- apply(res, 1, function(x) quantile(x, c(0.05, 0.20, 0.80, 0.95)))
```

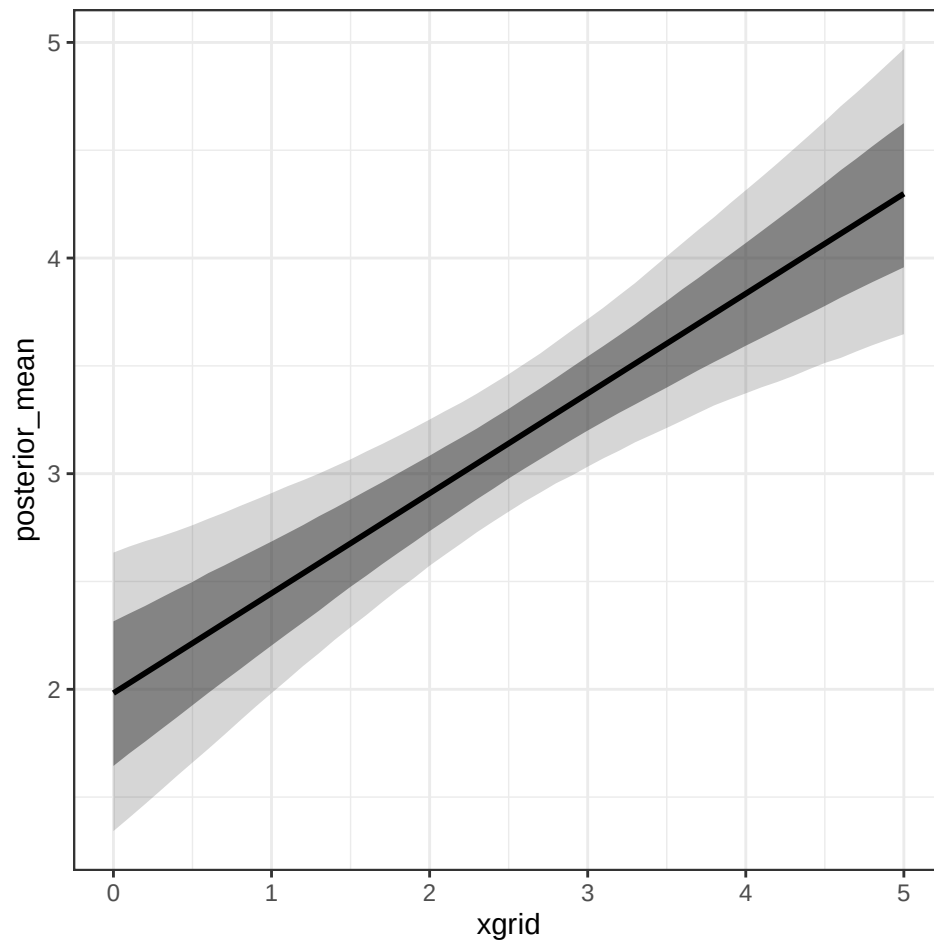
```
posterior_mean <- rowMeans(res)
```

```
#posterior_med <- apply(res, 1, median)
```

```

tibble(x=xgrid,
q05=quantiles[1, ],
q20=quantiles[2, ],
q80=quantiles[3, ],
q95=quantiles[4, ],
mean=posterior_mean) %>%
ggplot() +
geom_ribbon(aes(x=xgrid, ymin=q05, ymax=q95), alpha=0.2) +
geom_ribbon(aes(x=xgrid, ymin=q20, ymax=q80), alpha=0.5) +
geom_line(aes(x=xgrid, y=posterior_mean), size=1) +
theme_bw()

```



Hierarchical Modeling (Eight Schools Example)

The eight schools problem (Rubin 1981) considers the effectiveness of different SAT coaching programs conducted in parallel at eight schools. It has become a classic problem (Bayesian Data Analysis, Stan) that illustrates the usefulness of hierarchical modeling for sharing information between exchangeable groups.

We compute the average SAT score in those who did take the program minus those who did not take part and find the average difference varies by school. What accounts for these differences?

```

## data ##
schools_dat <- list(J = 8,
  y = c(28, 8, -3, 7, -1, 1, 18, 12),

```

```
sigma = c(15, 10, 16, 11, 9, 11, 10, 18))
```

Our options:

1. Estimate the effect of the program in every school independently: separate prior for each school effect.
2. Assume the effect is the same in each school: combine all of the data.
3. A compromise of the 2 options above?

Model Specification:

$$\theta_j \sim N(\mu, \tau^2)$$

$$y_j \sim N(\theta_j, \sigma_j^2)$$

- θ_j are the true unknown effects of the program in school j
- y_j is the observed effects of the program in school j
- Add a shared normal prior distribution to θ_j
- Assume the global mean, μ is also unknown
- τ^2 determines how much weight we put on the independent estimate vs the pooled estimate
 - if τ^2 is large, the prior for θ_j is not very strong: As $\tau^2 \rightarrow \infty$, equivalent to **no pooling**.
 - if τ^2 is small, we are assuming apriori that θ_j are all very close: As $\tau^2 \rightarrow 0$, equivalent to **complete pooling** model, $\theta_j = \mu$.

Estimate hyperparameter μ and τ with no pooling (option 1):

```
data {
  int<lower=0> J;           // # of schools
  array[J] real y;         // estimated treatment effects
  array[J] real<lower=0> sigma; // standard error of effect estimates
}

parameters {
  real theta[J];           // school effect
  real mu;                 // mean for schools
  real<lower=0> tau;        // variance between schools
}

model {
  theta ~ normal(mu, tau);
  y ~ normal(theta, sigma);
}
```

Estimate hyperparameter μ and τ with complete pooling (option 2):

```
data {
  array[J] real y;         // estimated treatment effects
  array[J] real<lower=0> sigma; // standard error of effect estimates
  real<lower=0> sigma[J];   // std err of effect
}

parameters {
  real theta;              // school effect
  real mu;                 // mean for schools
}
```

```

    real<lower=0> tau;    // variance between schools
  }

model {
  theta ~ normal(mu, tau);
  y ~ normal(theta, sigma);
}

```

Further, introduce η , the unscaled deviation from μ by school and let:

$$\theta_j = \mu + \tau * \eta_j,$$

where $\eta_j \sim N(0, 1)$

Noting that

$$\theta_j \mid \mu, \tau \sim N(\mu, \tau^2)$$

```

data {
  int<lower=0> J;          // number of schools
  array[J] real y;        // estimated treatment effects
  array[J] real<lower=0> sigma; // standard error of effect estimates
}
parameters {
  real mu;                // population treatment effect
  real<lower=0> tau;       // standard deviation in treatment effects
  vector[J] eta;          // unscaled deviation from mu by school
}
transformed parameters {
  vector[J] theta = mu + tau * eta; // school treatment effects
}
model {
  target += normal_lpdf(eta | 0, 1); // prior log-density
  target += normal_lpdf(y | theta, sigma); // log-likelihood
}

```

```

library(cmdstanr)
install_cmdstan()
stan_model <- cmdstan_model("eight_schools.stan")
stan_fit <- stan_model$sample(
  data = schools_dat,
  refresh = 0, show_messages=FALSE)
samples <- stan_fit$draws(format = "df")
head(samples)

```

```

## # A draws_df: 6 iterations, 1 chains, and 19 variables
##   lp__      mu tau eta[1] eta[2] eta[3] eta[4] eta[5]
## 1  -42   0.14 7.9  -0.34 -0.268  0.13    1.9 -0.243
## 2  -43   3.51 2.9  -0.63 -1.078  0.26    2.4 -0.213
## 3  -41  14.92 4.2   1.37  0.686 -0.60   -1.3 -0.291
## 4  -46  -3.38 0.9  -0.29  0.118  1.18    2.0 -0.279
## 5  -44  -1.45 1.9  -0.29 -0.046  0.99    2.1 -0.033
## 6  -41   7.58 2.7   1.01  0.233 -0.58   -1.9 -0.073
## # ... with 11 more variables
## # ... hidden reserved variables {'.chain', '.iteration', '.draw'}

```

Is the training program effective in school j ? Find $P(\theta_j > 0|y)$

```
mean(samples$`theta[1]`>0) # school 1
```

```
## [1] 0.949
```

```
mean(samples$`theta[5]`>0) # school 5
```

```
## [1] 0.8105
```

On average (over all schools) is the training program effective? Find $P(\mu > 0|y)$

```
mean(samples$mu>0)
```

```
## [1] 0.9425
```

Shrinkage plot

In the Bayesian estimation, shrinkage occurs as a result of hierarchical models. When parameters are modeled as exchangeable and given a proper prior, it induces some amount of shrinkage. Likewise, Bayesian models with non- or weakly-informative priors will produce similar to the MLE. But stronger priors can produce estimate much different estimates than MLE.

```
shrinkage_plot <- function(empirical, posterior_mean,
                           shrink_point=mean(posterior_mean)) {

  tibble(y=empirical, pm=posterior_mean) %>%
    ggplot() +
    geom_segment(aes(x=y, xend=pm, y=1, yend=0), linetype="dashed") +
    geom_point(aes(x=y, y=1)) +
    geom_point(aes(x=pm, y=0)) +
    theme_bw(base_size=14) +
    geom_vline(xintercept=shrink_point, color="blue", size=1.2) +
    ylab("") + xlab("Estimate") +
    scale_y_continuous(breaks=c(0, 1),
                      labels=c("Posterior Mean", "MLE"),
                      limits=c(0,1))

}
```

```
theta_post <- colMeans(samples[c(12:19)])
print(theta_post)
```

```
## theta[1] theta[2] theta[3] theta[4] theta[5] theta[6] theta[7] theta[8]
## 11.528256 7.909164 6.115706 7.611227 5.287744 6.202577 10.696481 8.544669
```

```
# shrinkage plot #
shrinkage_plot(schools_dat$y, theta_post)
```